

Sistem Terdistribusi

IF2228

03: Remote Procedure Calls (RPC)

September 2026

Sistem Terdistribusi

IF2228

Pengampu

Yoga Dwitya P.

Today...

■ Last Session:

- Networks- Part II
 - Networking Principles: Encapsulation, Routing, and Congestion Control
 - Transport Layer

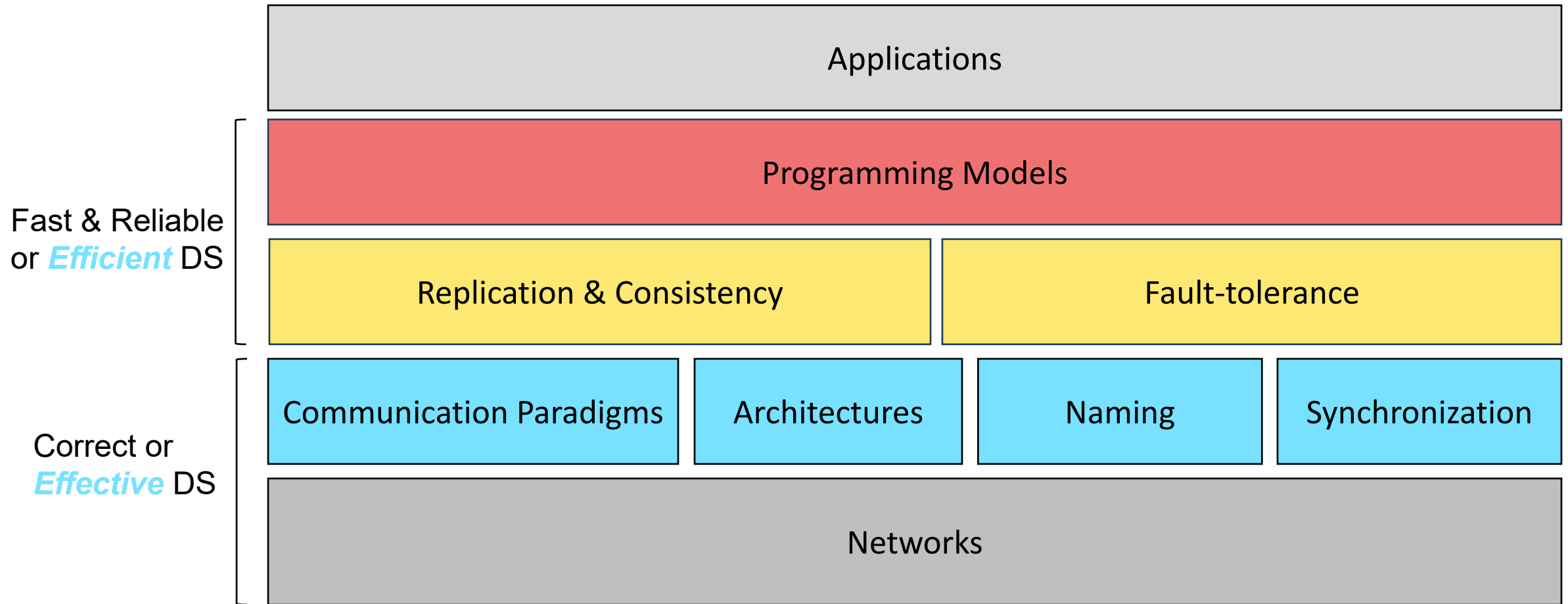
■ Today's Session:

- Remote Procedure Calls- Part I
 - IPC Primitives
 - Remote Invocations

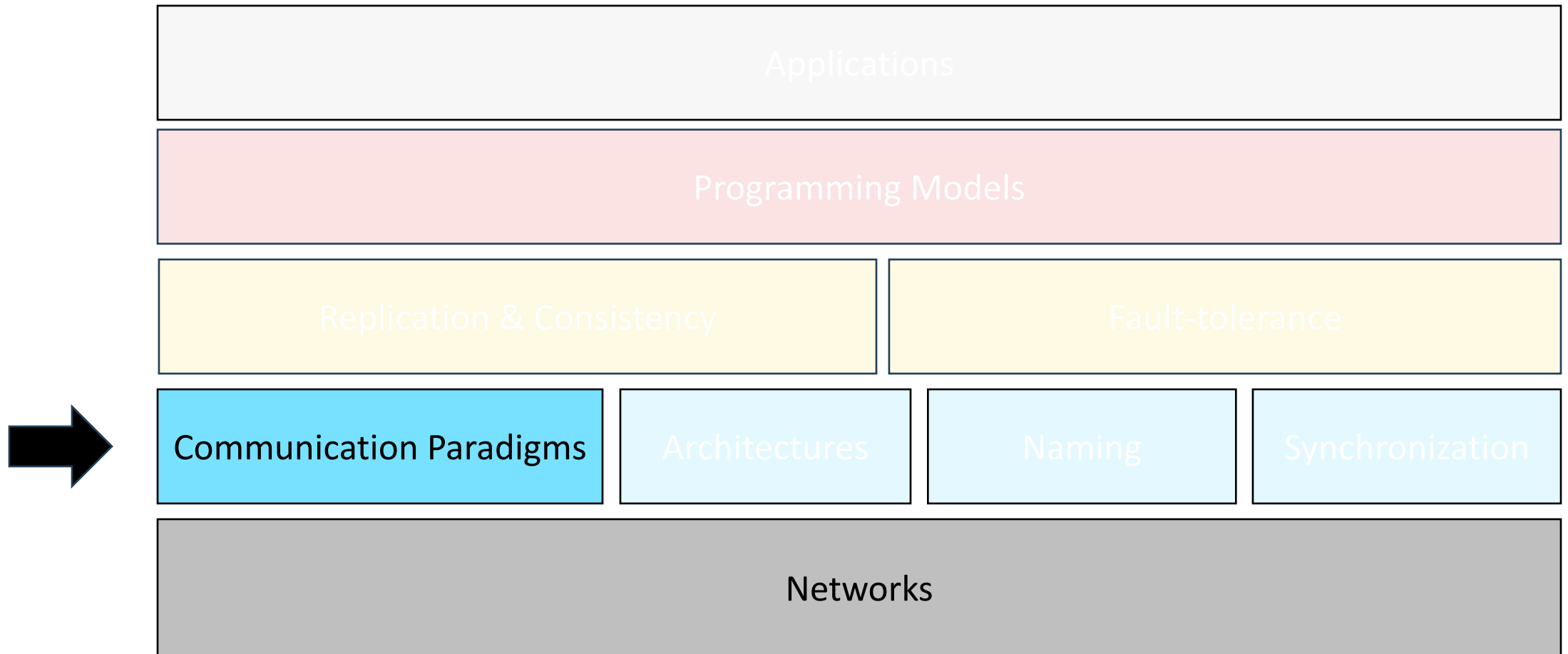
■ Announcement:

- PS1 is due on Sep 8 by midnight

Course Map



Course Map



Communicating Entities in Distributed Systems

- Communicating entities in distributed systems can be classified into two types:
 - System-oriented entities
 - Processes
 - Threads
 - Nodes
 - Problem-oriented entities
 - Objects (in *object-oriented programming* based approaches)

How can entities in distributed systems communicate?

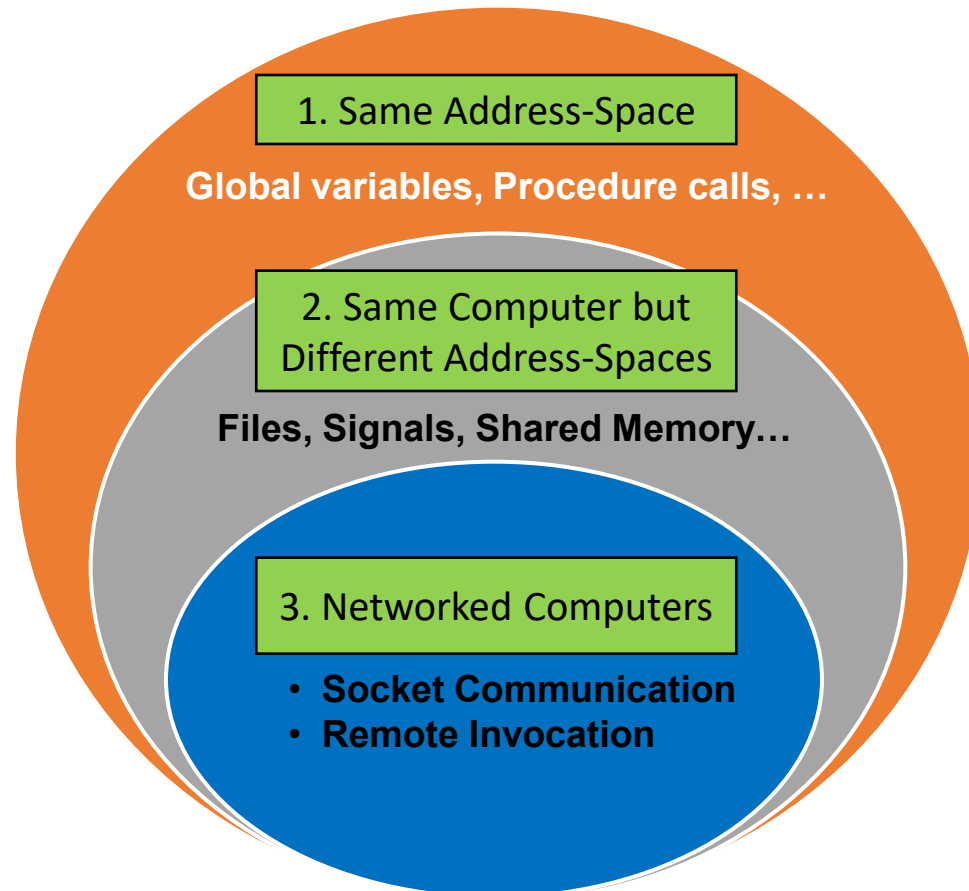


Communication Paradigms

- Communication paradigms describe and classify a set of methods by which entities can interact and exchange data

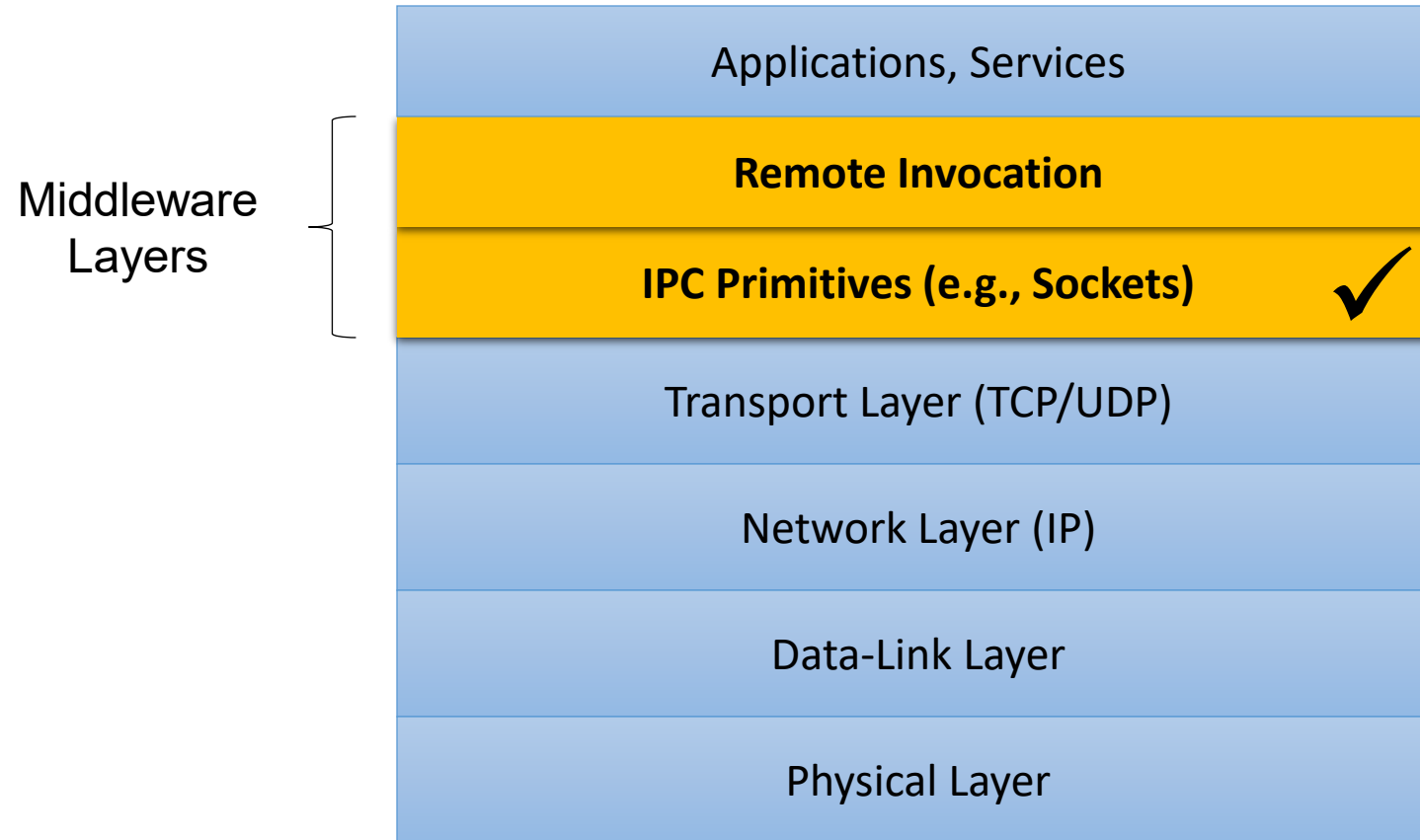
Classification of Communication Paradigms

- Communication paradigms can be categorized into *three* types based on where the entities reside. If entities are running on:



Today, we will study how entities that reside on **networked computers** communicate in distributed systems using socket communication and remote invocation

Middleware Layers

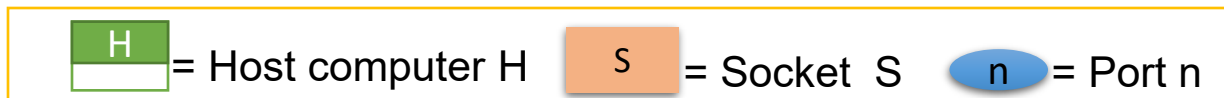


UDP Sockets

- UDP provides *connectionless* communication, with no acknowledgements or message retransmissions
- Communication mechanism:
 - Server opens a UDP socket *SS* at a known port *sp*,
 - Socket *SS* waits to receive a request
 - Client opens a UDP socket *CS* at a random port *cx*
 - Client socket *CS* sends a message to *ServerIP* and port *sp*
 - Server socket *SS* may send back data to *CS*



No ACK will be sent by the receiver

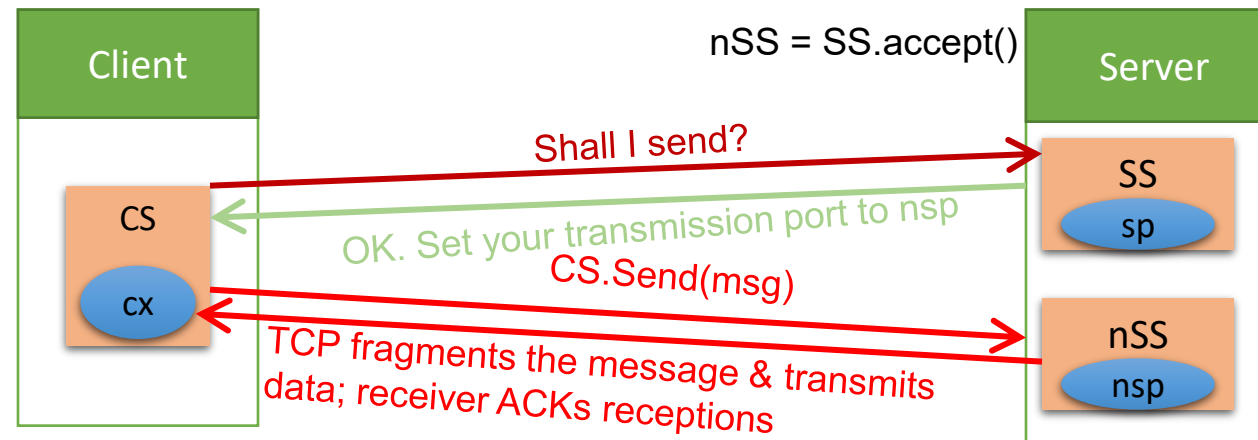


UDP– Design Considerations

- Sender must explicitly fragment a long message into smaller chunks before transmission
 - A maximum size of 548 bytes is suggested for transmission
- Messages may be delivered out-of-order
 - If necessary, programmer must re-order packets
- Communication is not reliable
 - Messages might be dropped due to check-sum errors or buffer overflows at routers
- Receiver should allocate a buffer that is big enough to fit the sender's message
 - Otherwise the message will be truncated

TCP Sockets

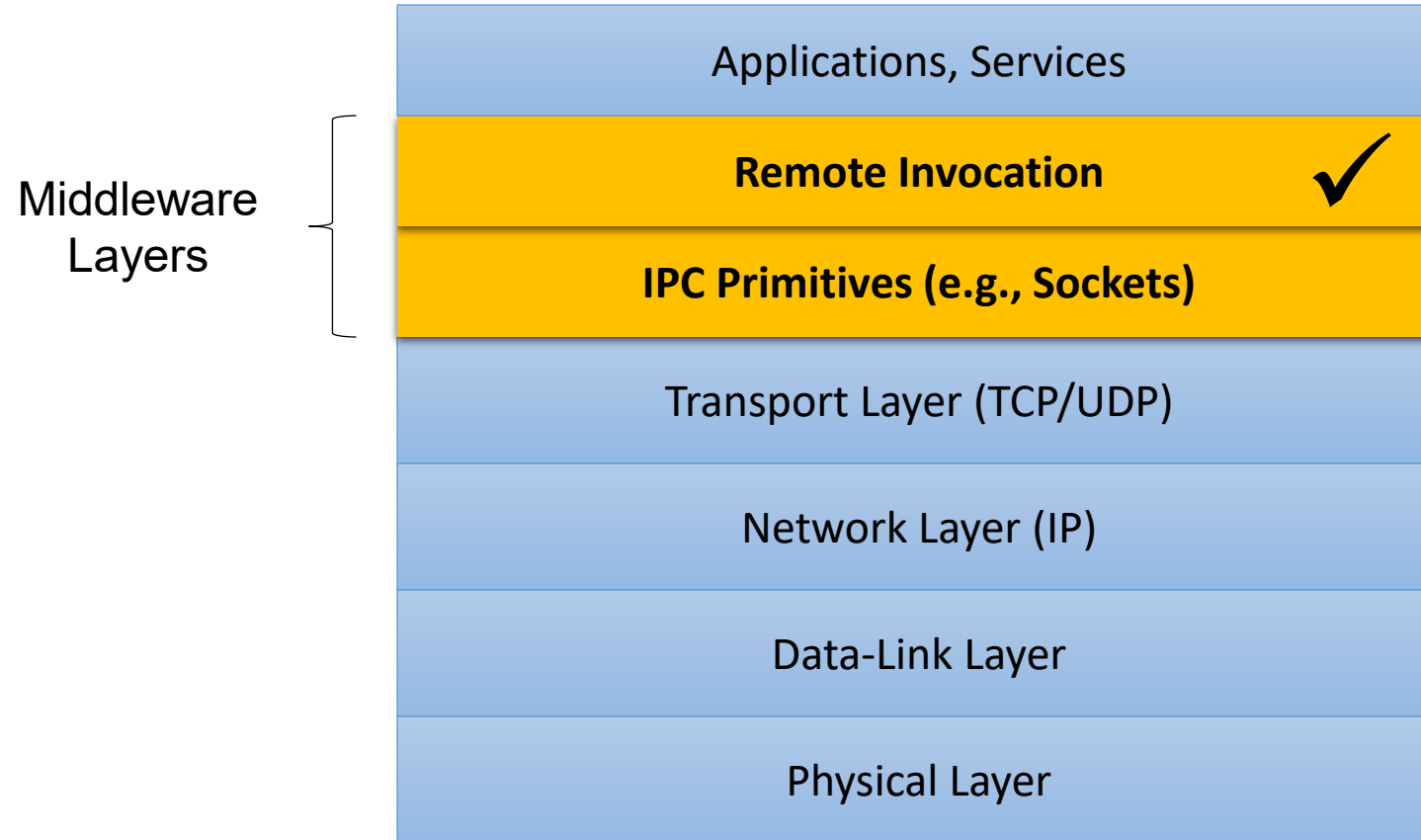
- TCP provides *in-order* delivery, *reliability*, and *congestion control*
- Communication mechanism:
 - Server opens a TCP server socket *SS* at a known port *sp*
 - Server waits to receive a request (using *accept* call)
 - Client opens a TCP socket *CS* at a random port *cx*
 - CS initiates a **connection initiation message** to ServerIP and port *sp*
 - Server socket *SS* allocates a **new socket** *NSS* on **random port** *nsp* for the client
 - CS can **send data** to *NSS*



Main Advantages of TCP

- TCP ensures in-order delivery of messages
- Applications can send messages of any size
- TCP ensures *reliable communication* via using acknowledgements and retransmissions
- Congestion control of TCP regulates sender rate, and thus prevents network overload

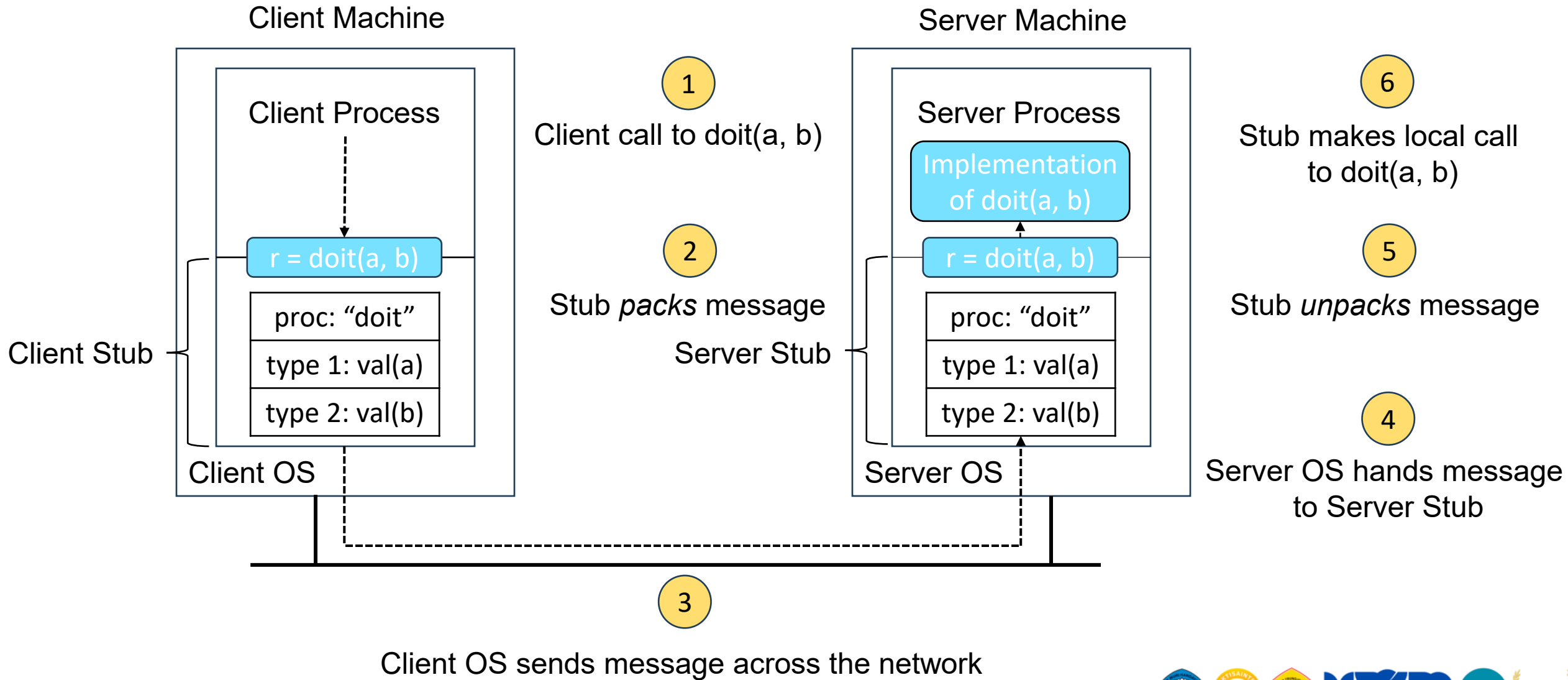
Middleware Layers



Remote Invocation

- Remote invocation enables an entity to call a procedure that typically executes on an another computer **without the programmer explicitly coding the details of communication**
 - The underlying middleware will take care of raw-communication
 - Programmer can *transparently* communicate with remote entity
- We will study two types of remote invocations:
 - a.Remote Procedure Calls (RPC)
 - b.Remote Method Invocation (RMI)

RPC



RPC: Summary

- The client procedure calls the client stub in the normal way
- The client stub *packs* a message and calls the local OS
- The client's OS sends the message to the remote OS
- The remote OS gives the message to the server stub
- The server stub *unpacks* the parameter(s) and calls the server
- The server does the work and returns the result to the stub
- The server stub *packs* the result in a message and calls its local OS
- The server's OS sends the message to the client's OS
- The client's OS gives the message to the client stub
- The client stub *unpacks* the result and returns it to the client



RPC: Example

```
import channel, pickle
```

```
class DBList:
```

```
    def append(self, data):  
        self.value.extend(data)  
        return self
```

```
class Client:
```

```
    def append(self, data, dbList):  
        msglst = (APPEND, data, dbList) # message payload  
        self.channel.sendTo(self.server, msglst) # send message to server  
        msgrcv = self.channel.recvFrom(self.server) # wait for an incoming msg  
  
        # A call to recvFrom returns a [senderID, message] pair  
        return msgrcv[1] # pass returned message to caller
```



RPC: Example

```
class Server:
    def append(self, data, dbList):
        return dbList.append(data)

    def run(self):
        while True:
            msgreq = self.channel.recvFromAny() # wait for any request
            client = msgreq[0] # see who is the caller
            msgrpc = msgreq[1] # fetch the actual request

            # At this point, msgreq should have the form
            # (operation, data, list)
            if APPEND == msgrpc[0]: # check what is being requested
                result = self.append(msgrpc[1], msgrpc[2]) # do local call
                self.channel.sendTo(client, result) # return response
```



Challenges in RPC

- Parameter passing
 - Procedure parameters and results have to be transferred over the network as bits
- Data representation
 - Data representation has to be uniform
 - Architecture of the sender and receiver machines may differ
- Failure Independence
 - Client and server might fail independently

Challenges in RPC

- Parameter passing
 - Procedure parameters and results have to be transferred over the network as bits
- Data representation
 - Data representation has to be uniform
 - Architecture of the sender and receiver machines may differ
- Failure Independence
 - Client and server might fail independently



Parameter Passing

- Packing parameters into a message that will be transmitted over the network is called *parameter marshalling*
- The parameters of the procedure and the result have to be marshaled before transmitting them over the network
- Two types of parameters can be passed:
 1. Value parameters
 2. Reference parameters

1. Passing Value Parameters

- Value parameters have complete information about the variable, and can be directly encoded into the message
 - E.g., integer, float, character
- Values are passed through **call-by-value**
 - The changes made by the callee procedure are not reflected in the caller procedure

2. Passing Reference Parameters

- Passing *reference* parameters like *value* parameters in RPC leads to incorrect results due to two reasons:
 - a. Invalidity of reference parameters at the server
 - Reference parameters are valid only within client's address space
 - **Solution:** Pass the reference parameter by copying the data that is referenced
 - b. Changes to reference parameters are not reflected back at the client
 - **Solution:** "Copy/Restore" the data
 - Copy-back the value at server to the client



RPC: Example *Revised*

```
import channel, pickle
```

```
class DBList:
```

```
def append(self, data):
    self.value.extend(data)
    return self
```

```
class Client:
```

```
def append(self, data, dbList):
    msglst = (APPEND, data, dbList) # message payload
    msgsnd = pickle.dumps(msglst) # serialize message
    self.channel.sendTo(self.server, msgsnd) # send message to server
    msgrcv = self.channel.recvFrom(self.server) # wait for an incoming msg
    retval = pickle.loads(msgrcv[1]) # deserialize message

    return retval # pass returned message to caller
```

RPC: Example *Revised*

```
class Server:
    def append(self, data, dbList):
        return dbList.append(data)

    def run(self):
        while True:
            msgreq = self.channel.recvFromAny() # wait for any request
            client = msgreq[0] # see who is the caller
            msgrpc = pickle.loads(msgreq[1]) # deserialize message

            if APPEND == msgrpc[0]: # check what is being requested
                result = self.append(msgrpc[1], msgrpc[2]) # do local call
                msgres = pickle.dumps(result) #serialize response
                self.channel.sendTo(client, msgres) # return response
```



Challenges in RPC

- Parameter passing
 - Procedure parameters and results have to be transferred over the network as bits
- Data representation
 - Data representation has to be uniform
 - Architecture of the sender and receiver machines may differ
- Failure Independence
 - Client and server might fail independently

Data Representation

- Computers in DSs often have different architectures and operating systems
 - The size of the data-type differ
 - E.g., A *long* data-type is 4-bytes in 32-bit Unix, while it is 8-bytes in 64-bit Unix systems
 - The format in which the data is stored differs
 - E.g., Intel stores data in little-endian format, while SPARC stores in big-endian format
- The client and server have to agree on how simple data is represented in the message
 - E.g., Format and size of data-types such as integer, char and float



Example: Stub Generation

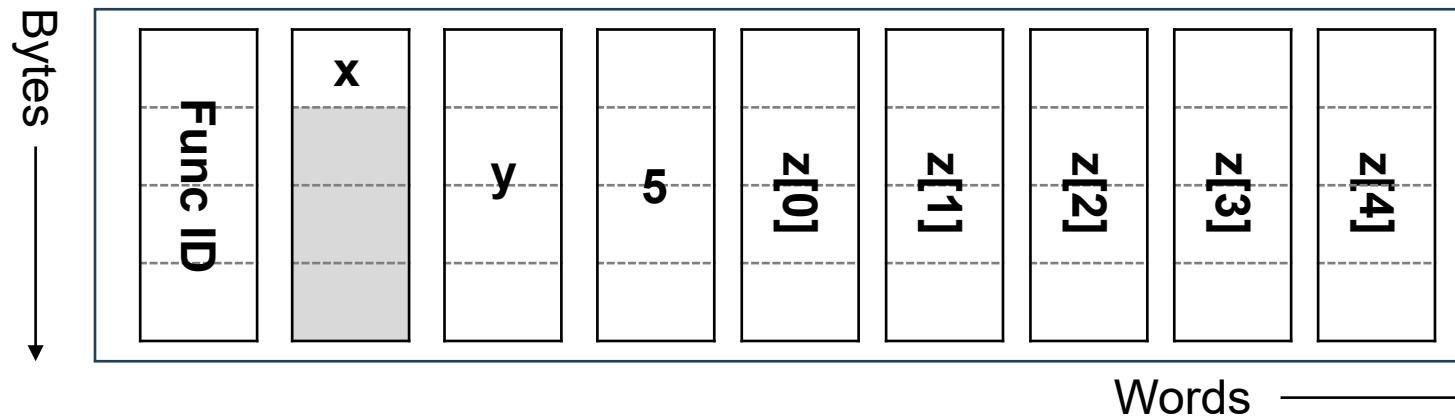
- To this end, RPC requires that the *caller* and *callee* agree on:
 1. The format of the messages they exchange
 2. The representations of data structures
 3. The actual exchange of messages
- In other words, the caller and callee should follow the same protocol or RPC will not work correctly
- Consider the following function:

void someFunction(char x, float y, int z[5])

Example: Stub Generation

1. Assuming a word is four bytes, the RPC protocol might prescribe that:
 - Character **x** should be transmitted in the rightmost byte of a word
 - Float **y** should be transmitted as a whole word
 - Array **z** should be transmitted as a group of words whose size is equal to the array length, preceded by a word giving the length

Both, the **Client Stub** and **Server Stub** will use the same format



void someFunction(char x, float y, int z[5])

Example: Stub Generation

2. In addition, the RPC protocol might prescribe that:
 - Integers are represented in two's complement
 - Characters are represented in 16-bit Unicode
 - Floats are represented in the IEEE standard #754 format
 - Everything is stored in little endian
3. Lastly, the RPC protocol might prescribe that messages are exchanged using:
 - A connection-oriented transport service (e.g., TCP)
 - A connectionless transport service (e.g., UDP)

Challenges in RPC

- Parameter passing
 - Procedure parameters and results have to be transferred over the network as bits
- Data representation
 - Data representation has to be uniform
 - Architecture of the sender and receiver machines may differ
- Failure Independence
 - Client and server might fail independently

Failure Independence

- In the local case, the client and server live or die together
- In the remote case, the client sees new *failure types* (*more on this next lecture*)
 - Network failure
 - Server machine crash
 - Server process crash
- Thus, failure handling code has to be more thorough (and essentially more complex)

Next Lecture...

- Remote Procedure Calls- Part II

Today...

- Last Session:

- RPC- Part I

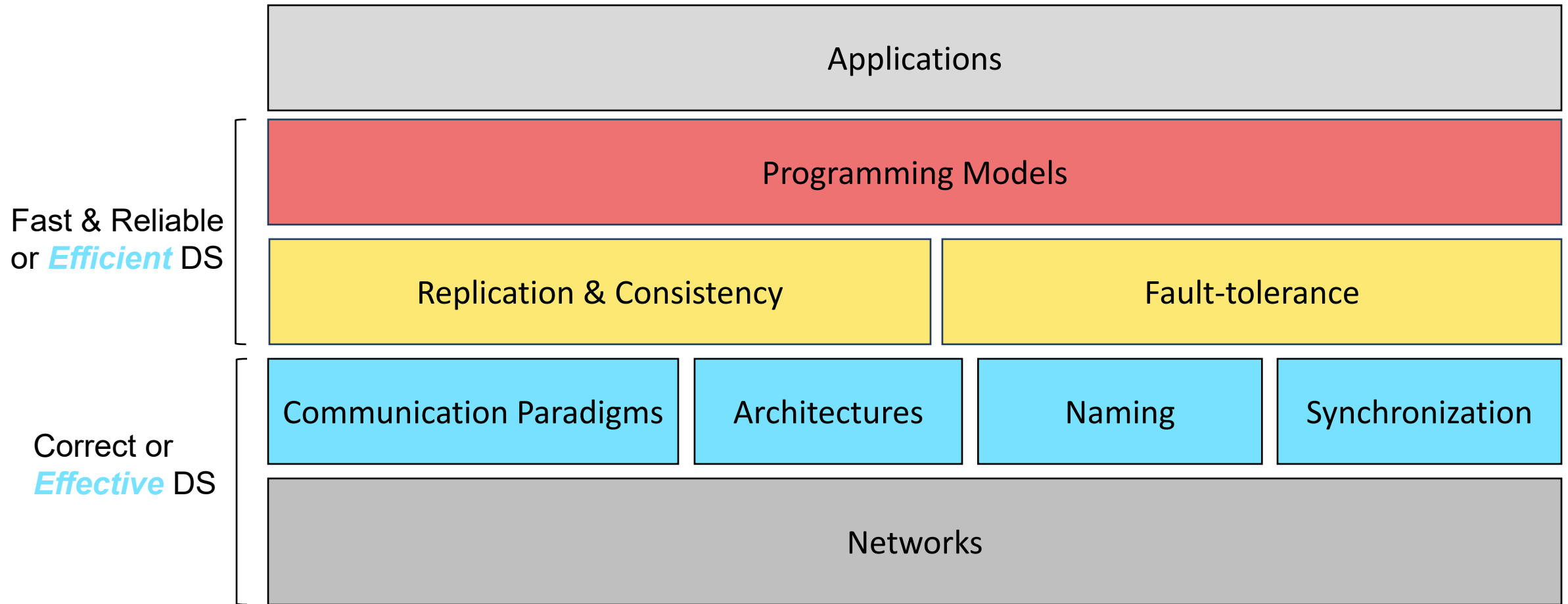
- Today's Session:

- Continue with Remote Procedure Calls

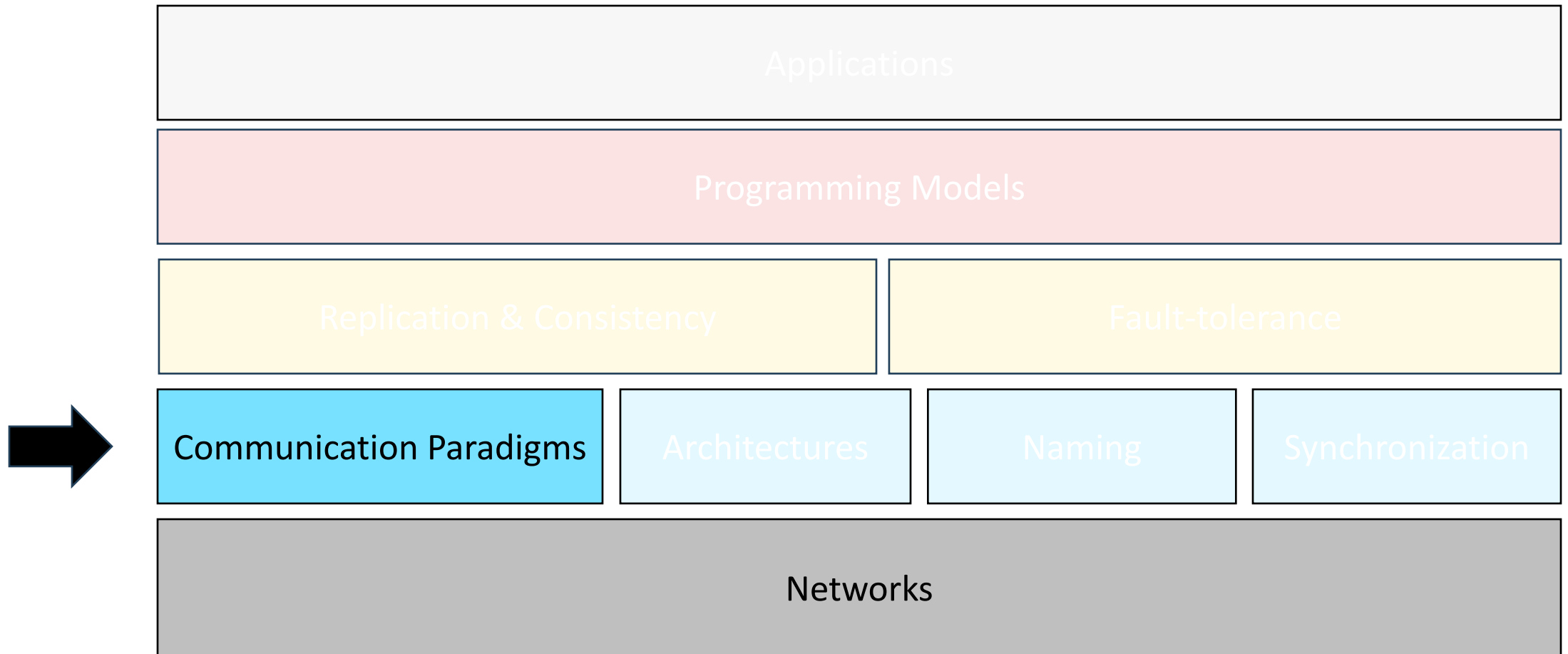
- Announcements:

- Quiz I is on Tuesday, September 10
- Project I will be out today. It is due on Oct 6th (design report due on September 22)

Course Map

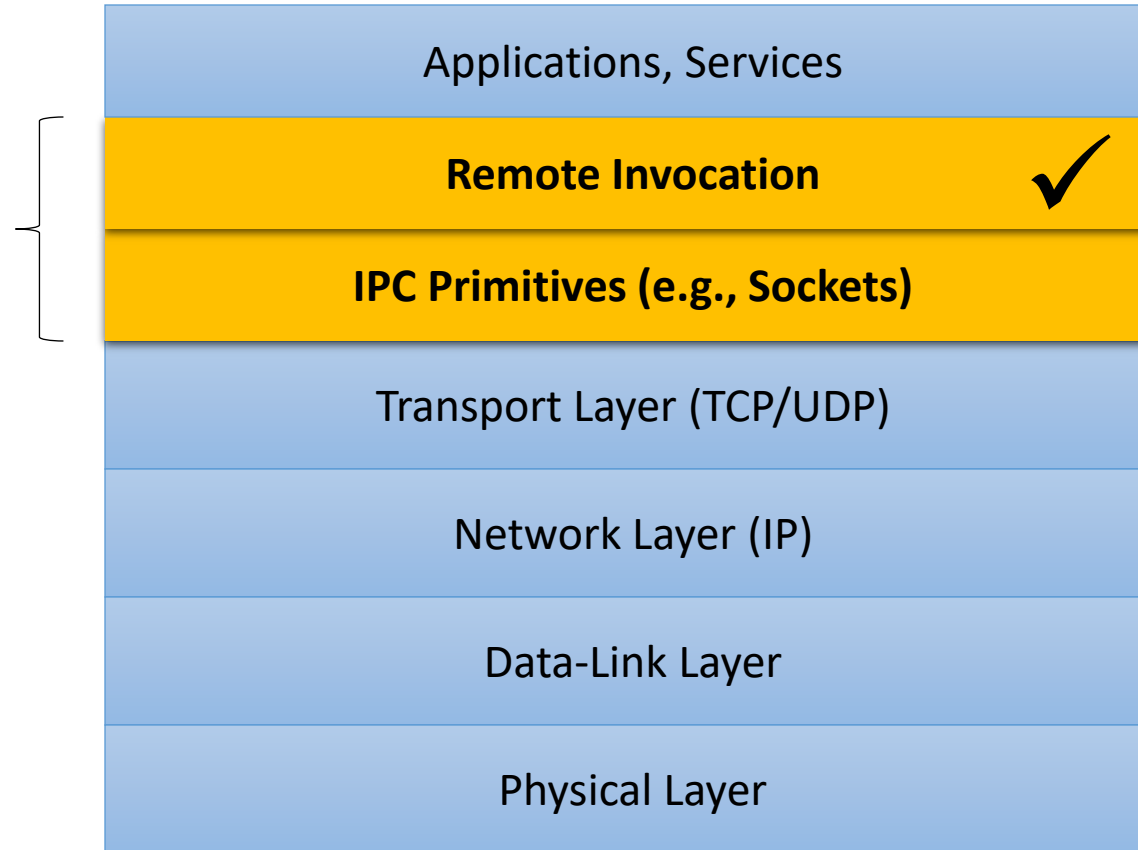


Course Map

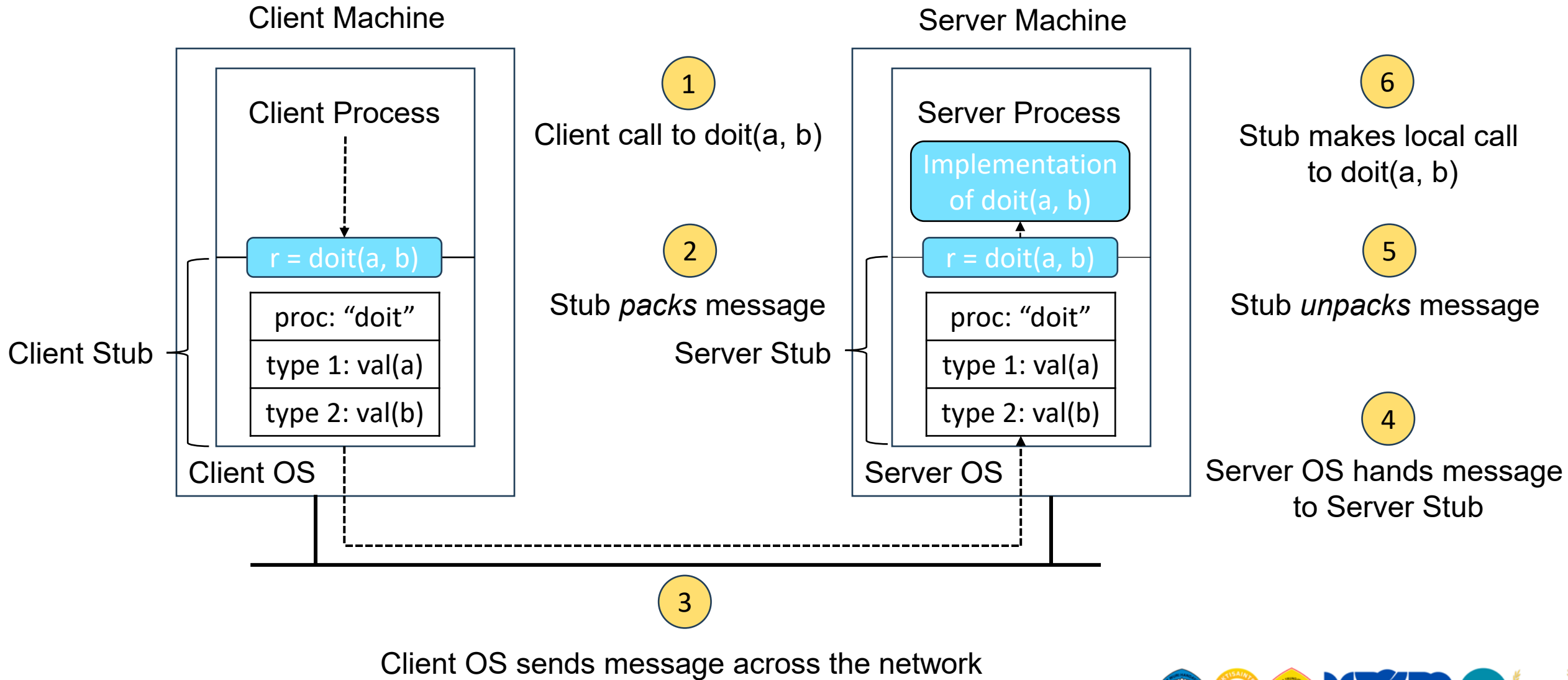


Middleware Layers

Middleware
Layers



RPC



Challenges in RPC

- Parameter passing
 - Procedure parameters and results have to be transferred over the network as bits
- Data representation
 - Data representation has to be uniform
 - Architecture of the sender and receiver machines may differ
- Failure Independence
 - Client and server might fail independently



Failure Independence

- In the local case, the client and server live or die together
- In the remote case, the client sees new failure types
 - Network failure (can it be handled by TCP? -- *more on this later*)
 - Server failure, which could itself be of many types
- Thus, failure handling code has to be more thorough (and essentially more complex)
 - *Retransmission* and *timeout* mechanisms
 - *Atomicity* and *durability* guarantees

Server Failure Types

- RPC systems may suffer from various types of server failures

Type of Failure	Description
• Crash Failure	• A server halts, but was working correctly until it stopped
• Omission Failure • Receive Omission • Send Omission	• A server fails to respond to incoming requests • A server fails to receive incoming messages • A server fails to send messages
• Timing Failure	• A server's response lies outside the specified time interval
• Response Failure • Value Failure • State Transition Failure	• A server's response is incorrect • The value of the response is wrong • The server deviates from the correct flow of control
• Byzantine Failure	• A server may produce arbitrary responses at arbitrary times

Handling Failures: Timeout Mechanism

- To allow for occasions where a request or a reply message is lost, RPC can use a **timeout mechanism**
- There are various options as to what RPC can do after a timeout:
 - Either return immediately with an indication to the client that the request has failed
 - Or *retransmit* the request repeatedly until either a reply is received or the server is assumed to have failed
- How to pick a timeout value?
 - At best, use empirical/theoretical statistics
 - At worst, no good value exists

Idempotent Operations

- In cases when the request message is retransmitted, the server may receive it *more than once*
- This can cause an operation to be executed more than once for the same request
- Caveat: Not every operation can be executed more than once and obtain the same result each time!
- Operations that CAN be executed repeatedly with the same effect are called *idempotent operations*

Duplicate Filtering

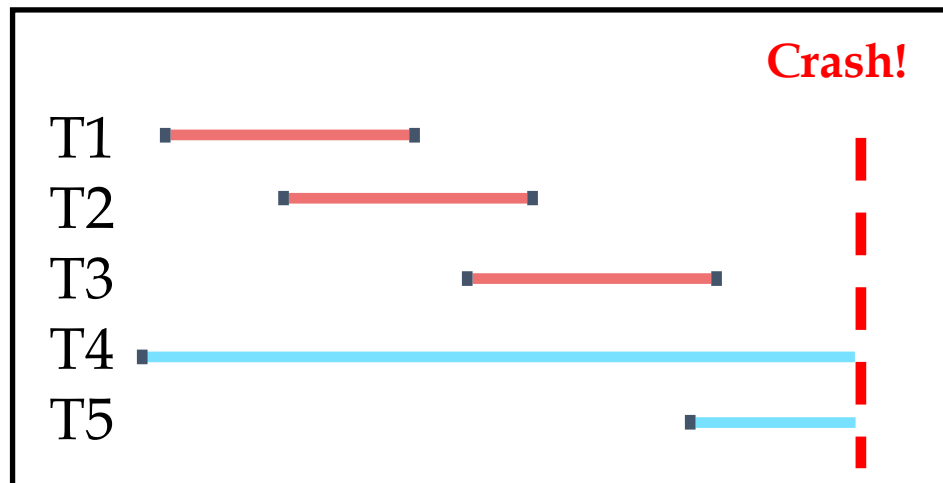
- To avoid problems with operations, the server should:
 - Identify successive messages from the “same” client
 - Monotonically increasing *sequence numbers* can be used
 - Filter out duplicates
- Upon receiving a “duplicate” request, the server can:
 - Either *re-execute* the operation again and reply
 - Possible only for idempotent operations
 - Or *avoid re-executing* the operation via *retaining* its output in a non-volatile history (or *log*) file
 - Might necessitate ensuring *atomicity* and *durability*

Atomicity and Durability

- What is **atomicity**?
 - A property where either all the actions of a *transaction* (i.e., the request) are carried out or none at all
- What is **durability**?
 - A property where if a *transaction* commits, its effects persist (even if the server crashes before all its changes are reflected on disk)
- How can **atomicity** and **durability** be ensured?

Ensuring Atomicity and Durability

- A server can ensure:
 - *Atomicity* by *undoing* the actions of transactions *that did not commit*
 - *Durability* by *redoing* (all) the actions of *committed* transactions
 - **Checkpointing** can be used to avoid redoing all actions



- Desired Behavior after the system restarts:
 - T1, T2 & T3 should be durable (i.e., *redone*)– if **checkpointing** is used, maybe not all need to be redone
 - T4 & T5 should be rolled back (i.e., *undone*)

Stealing Frames and Forcing Pages

- To realize what it takes to ensure *atomicity* and *durability*, it is necessary to understand what happens during normal execution
 - Can the changes made to an object **O** in memory by a transaction **T** be written to disk before **T** commits?
 - Yes, if another transaction steals **O**'s frame (a *steal approach* is said to be in place)
 - No, if stealing is not allowed (a *no-steal approach* is said to be in place)
 - When **T** commits, must we ensure that all its changes are immediately *forced* to disk?
 - Yes, if a *force approach* is used
 - No, if a *no-force approach* is used

Steal vs. No-Steal & Force vs. No-Force Approaches

- What if a *no-steal approach* is used?
 - We do not have to *undo* the changes of an aborted transaction (+)
 - But this assumes that all pages modified by running transactions can be accommodated in memory (-)
- What if a *force approach* is used?
 - We do not have to *redo* the changes of a committed transaction (+)
 - But this may cause high I/O cost (e.g., when a highly used page is updated in succession by 20 transactions, it would be written to disk 20 times!) (-)

Steal vs. No-Steal & Force vs. No-Force Approaches

- To this end, we can think of *four* alternatives:

	No-Steal	Steal
Force	Trivial, but undesired	High I/O cost, but modified pages need not fit in memory
No-Force	Low I/O cost, but modified pages need to fit in memory	Low I/O cost, and modified pages need not fit in memory



- Most systems use a *steal, no-force approach*
 - More on *transaction semantics* later in the course!

Implementation Choices

- RPC transport can be implemented in different ways to provide different *delivery guarantees*. The main choices are:
 1. **Retry request service** (*client side*): Controls whether to retransmit the request service until either a reply is received or the server is assumed to have failed
 2. **Duplicate filtering** (*server side*): Controls when retransmissions are used and whether to filter out duplicate requests at the server
 3. **Retention of results** (*server side*): Controls whether to keep a history of result messages so as to enable lost replies to be retransmitted without re-executing the operations at the server

RPC Call Semantics

- Combinations of measures lead to a variety of possible *semantics* for the reliability of RPC

Fault Tolerance Measure			Call Semantics (Pertaining to Remote Procedures)
Retransmit Request Message	Duplicate Filtering	Re-execute Procedure or Retransmit Reply	
No	N/A	N/A	Maybe
Yes	No	Re-execute Procedure	At-least-once
Yes	Yes	Retransmit Reply	At-most-once

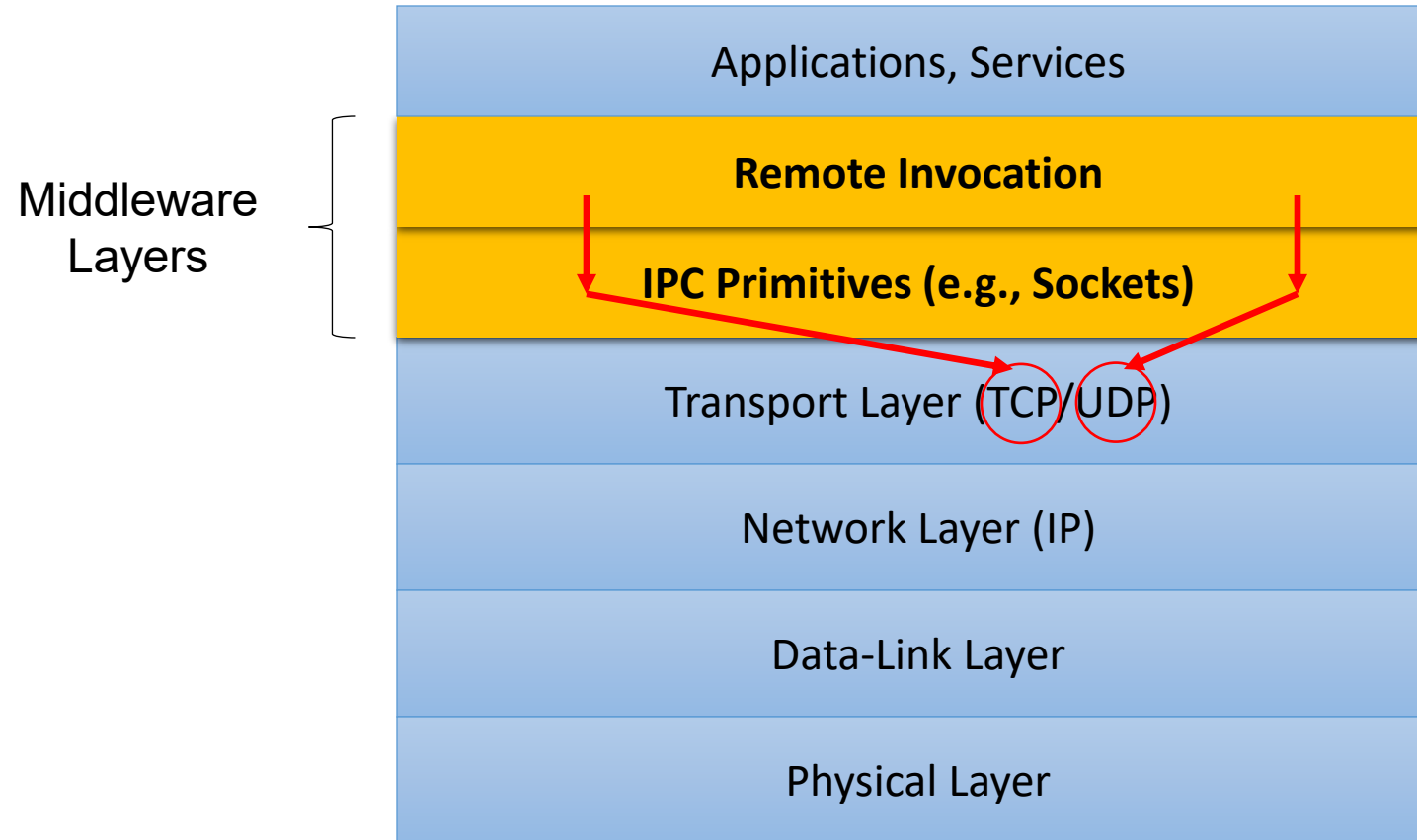
Ideally, we would want an *exactly-once* semantic!

RPC Failure Handling: Summary

- A client sends a request for a transaction to the server
 - If the request is *eventually* received by the server
 - One or more procedures will be triggered for the transaction
 - For every procedure, the server figures out if the procedure is *idempotent* or *non-idempotent*
 - If the procedure is idempotent
 - The server executes the procedure (i.e., uses *at-least-once semantic*)
 - Else:
 - The server checks if the procedure has been executed before for this transaction
 - If yes:
 - The sever reads and returns the result (i.e., uses *at-most-once semantic*) from non-volatile memory
 - If no:
 - The server executes the procedure and saves the result, while ensuring *atomicity* and *durability*



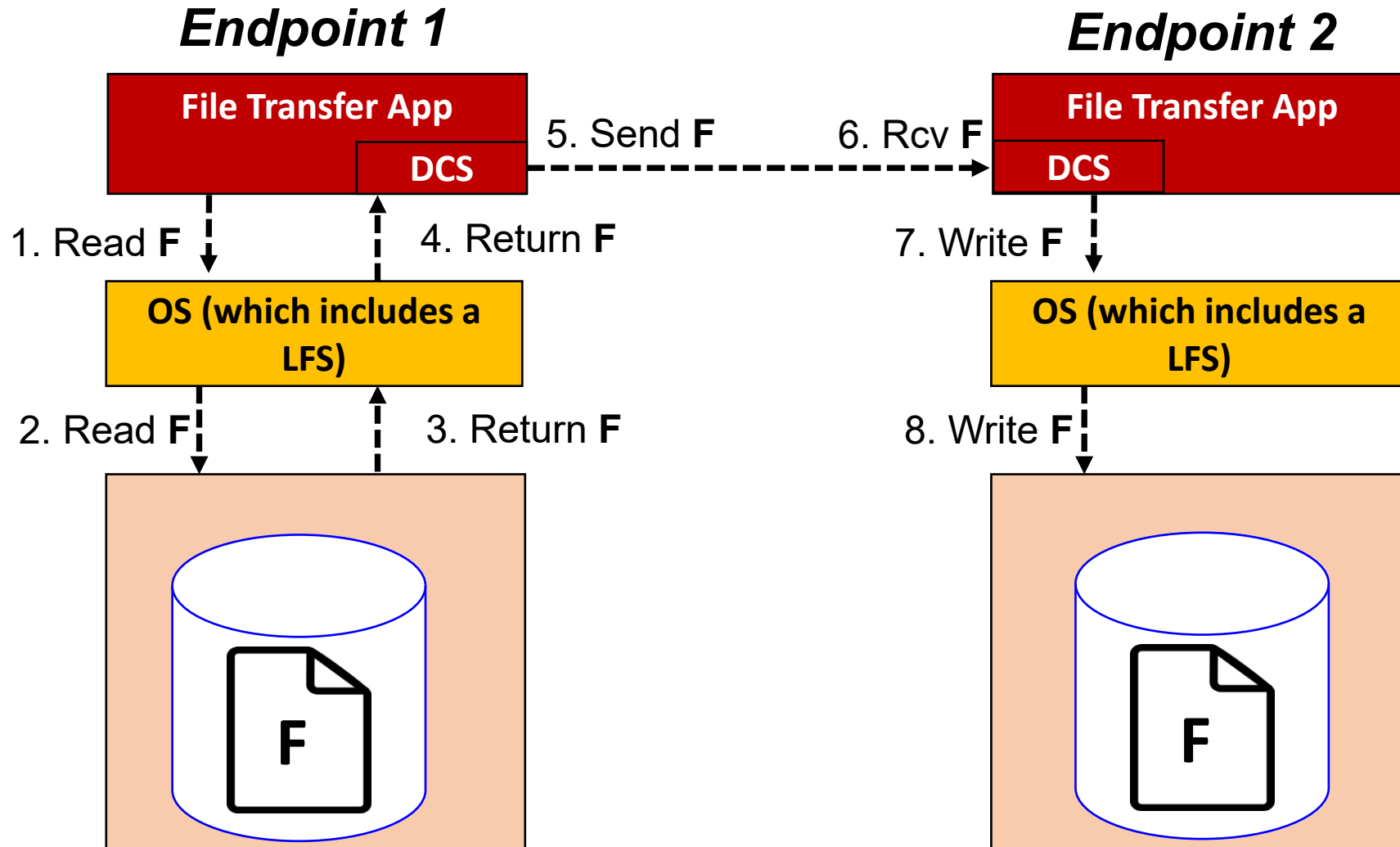
Middleware Layers



RPC over UDP or TCP

- If RPC is layered on top of UDP
 - Retransmission shall/can be handled by RPC
- If RPC is layered on top of TCP
 - Retransmission will be handled by TCP
 - Is it still necessary to take fault-tolerance measures within RPC?
 - Yes-- “End-to-End Arguments in System Design” by Saltzer *et. al.*

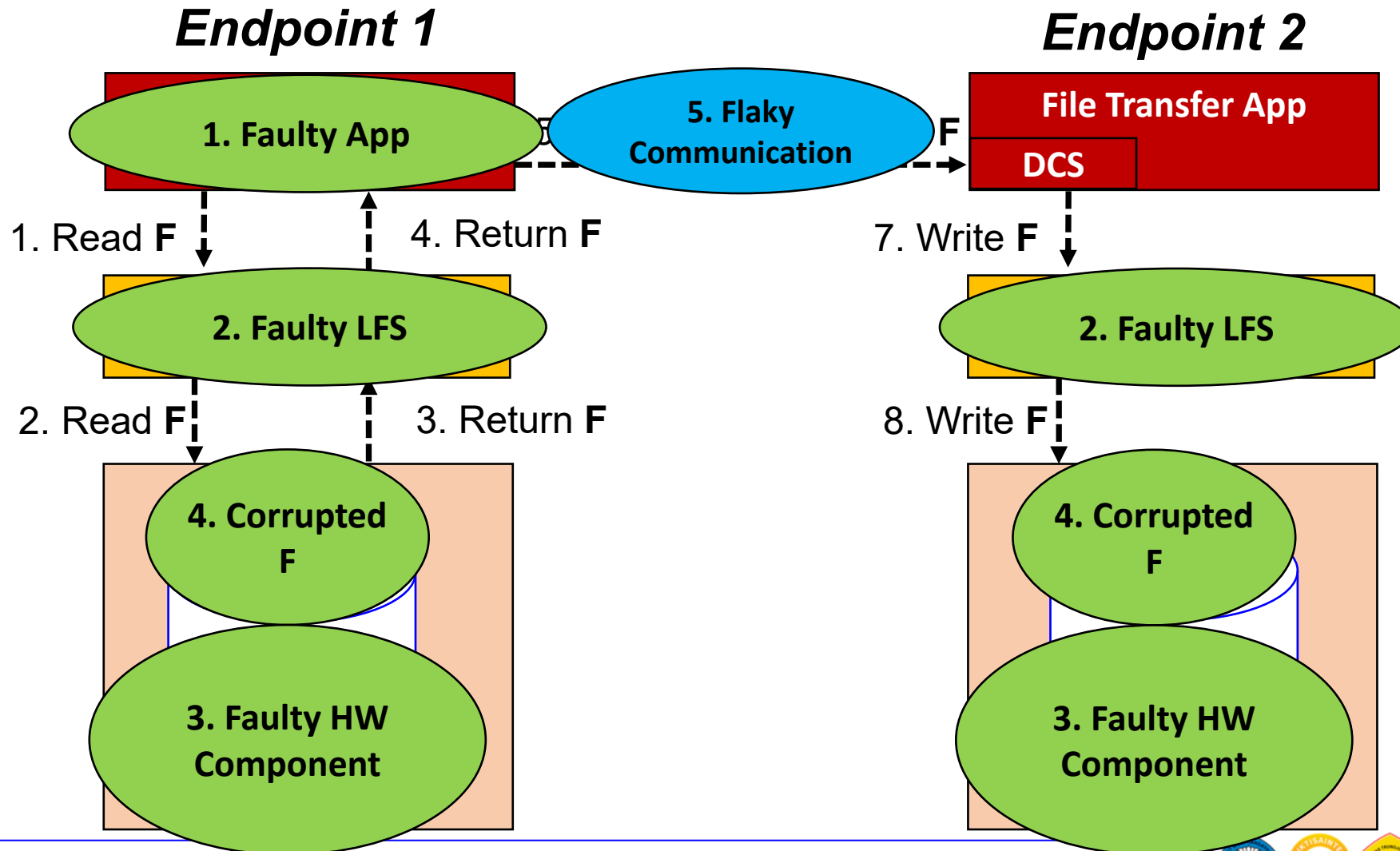
Careful File Transfer: Flow



DCS = Data Communication System; LFS = Local File System

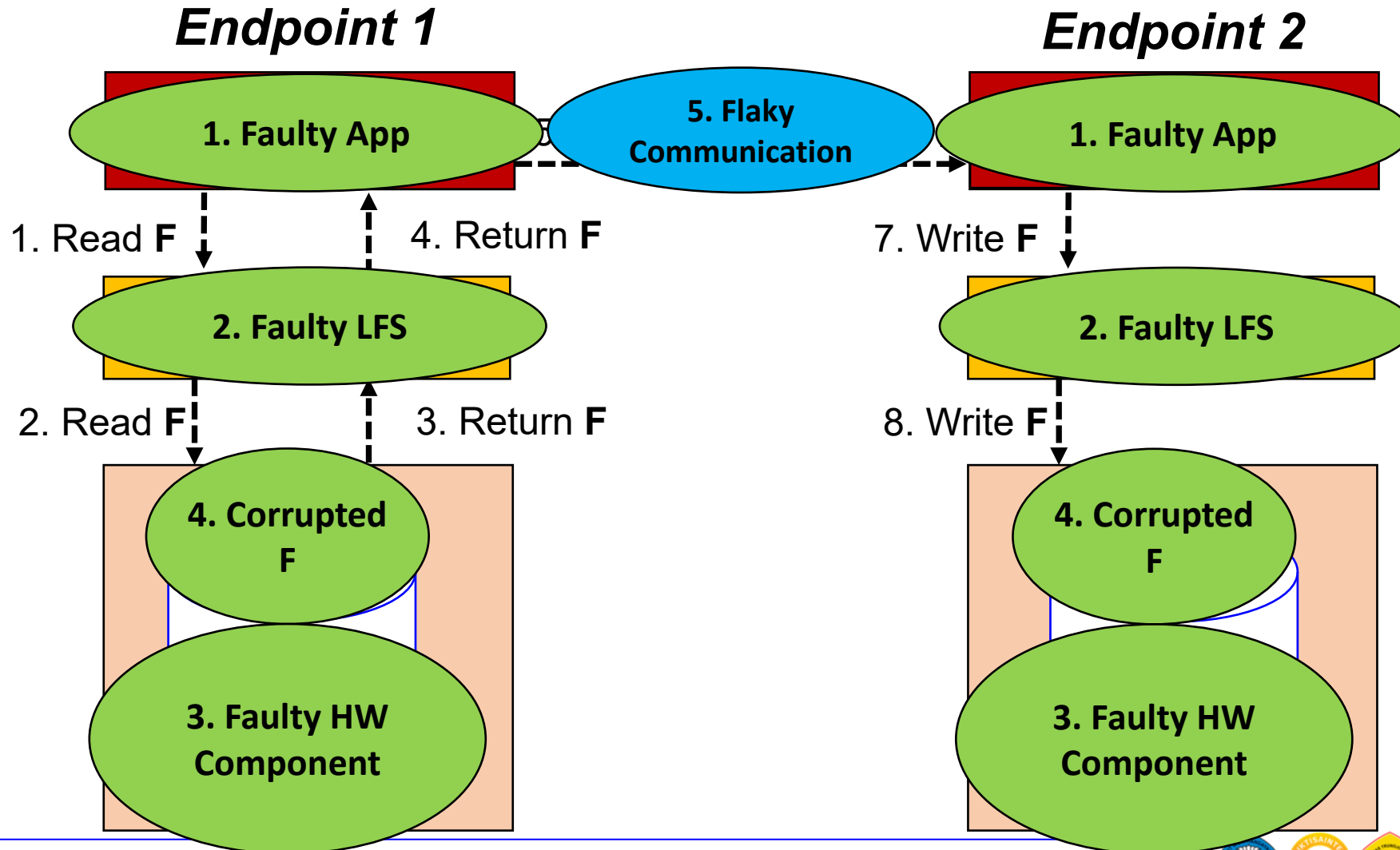


Careful File Transfer: Flow



DCS = Data Communication System; LFS = Local File System

Careful File Transfer: Possible Threats



DCS = Data Communication System; LFS = Local File System

Careful File Transfer: End-To-End Check and Retry

- Endpoint 1 stores with F a checksum C_A
- After Endpoint 2 writes F , it reads it again from disk, calculates a checksum C_B , and sends it back to Endpoint 1
- Endpoint 1 compares C_A and C_B
 - If $C_A = C_B$, commit the file transfer
 - Else, retry the file transfer

Careful File Transfer: End-To-End Check and Retry

- How many retries?
 - Usually 1 if failures are rare
 - 3 retries might indicate that some part of the system needs repairing
- What if the Data Communication System uses TCP?
 - Only threat 5 (e.g., packet loss due to a flaky communication) is eliminated
 - The frequency of retries gets reduced if the fault was caused by the communication system
 - More *control* traffic, but only missing parts of F need to be reshipped
 - The file transfer application still needs to apply *end-to-end reliability measures*!

Careful File Transfer: End-To-End Check and Retry

- What if the Data Communication System uses UDP?
 - Threat 5 (e.g., packet loss due to a flaky communication) is NOT eliminated- *F needs to be reshipped by the application if no measures are taken to address this threat*
 - The frequency of retries might increase
 - Worse performance on flaky links
 - The file transfer application still needs to apply *end-to-end reliability measures!*

In *both cases*, the application needs to provide end-to-end reliability guarantees!

Next Class...

- Architectures